

Step 1 Submission — Codebase Exploration and Experimental Verification

Replication of LoMix (NeurIPS 2025) on Synapse Multi-organ

Md. Irtiza Hossain

M.S. Computer Science & Engineering, BRAC University

mohammad.irtiza.hossain@gmail.com

7 September 2026

Paper Md Mostafijur Rahman, Radu Marculescu. *LoMix: Learnable Weighted Multi-Scale Logits Mixing for Medical Image Segmentation*. NeurIPS 2025.
Code github.com/SLDGroup/LoMix @ commit 464a945
Data Synapse Multi-organ (BTCV), 9 classes, 224×224 , authors' archive
Hardware GTX 960, 4 GB, capability 5.2 (Maxwell, no tensor cores)

This document covers both Step 1 deliverables: the experimental execution summary with setup notes (§3–§4, §9) and the technical report on the paper's contributions, observed bottlenecks and replication findings (§1–§2, §5–§8). Every number came out of a run on this machine, and the log that produced it is in `logs/`. Nothing is copied from the paper into a results table; where a run did not produce a number, the table says so.

Contents

1	What the paper does	1
2	Why I chose this paper	2
3	Environment and setup notes	2
4	Deviations from the paper's protocol	3
5	Results	3
5.1	Efficiency — the zero-inference-overhead claim holds	3
5.2	What fits in 4 GB	3
5.3	Training cost of the supervision, which the paper does not report	4
5.4	The supervision ablation	5
5.5	The <code>lomix</code> arm diverged	5
5.6	Learned mixing weights	5
6	Defects found in the released codebase	6
6.1	The training script cannot start	6
6.2	<code>mul</code> is numerically unbounded	6
6.3	<code>concat</code> rebuilds a random convolution on every forward pass	6
6.4	Training continues in eval mode after the first validation	7
6.5	Windows portability	7
7	Limitations of this replication	7

8	Discrepancies and what would settle them	7
9	Where I would take it	7
10	Reproducing this	8
	Appendix — slide outline	8

1 What the paper does

A multi-scale decoder emits one logit map per stage. Classical deep supervision applies a loss to each and sums them with fixed, usually uniform weights. LoMix changes two things.

Mixing. It supervises not only the individual maps but *combinations* of them. For four decoder outputs it enumerates all 11 subsets of size ≥ 2 and fuses each subset under four operations — add, mul, wf (a small learned fusion module) and concat — producing 44 synthetic logit maps, each receiving its own CE + Dice loss.

Learned weights. Every individual map and every mixed map contributes through a learned scalar passed through `softplus`, optimised jointly with the network by the same AdamW step.

```

encoder -- decoder ---+-- p4 ---+
                    |-- p3 --|   subsets (11) x ops (add, mul, wf, concat)
                    |-- p2 --|   |
                    +-- p1 ---+   v
                    |           44 mixed maps -- CE+Dice -- x softplus(w_op,subset)
                    |                                   |
                    +----- CE+Dice -- x softplus(w_i) -----+--> total loss

inference: encoder -- decoder -- p1           (mixing lives only in the loss)

```

The property that makes this attractive in practice: **all of it lives in the loss**. The mixing weights are parameters of `CombinatorialMutationsLossModule`, not of `EMCADNet`. At test time the network is unchanged — same graph, same FLOPs, same latency. Training cost rises; inference cost does not.

2 Why I chose this paper

It asks a question my own work keeps running into, in a different domain. **CLEAR-MoE** (under review) extracts sparse experts from a *frozen* Vision Transformer and selects which layers to convert using a calibration-driven criterion — a decision about which internal signals to trust. **Predictive Self-Conditioning** (under review, WACV 2027) feeds a LiDAR detector’s own predictions back as conditioning and is evaluated on *calibration*, not accuracy alone, because a self-conditioning loop that trusts a wrong prediction compounds its own error. LoMix asks the same question at the supervision level: *which of my own multi-scale outputs deserve weight?* It answers with a learned scalar.

It is also the axis the codebase exposes directly. `--supervision {last_layer, deep_supervision, mutation, lomix}` runs over one fixed network, so all four arms share the same architecture, data and schedule and differ only in the loss — a ready-made ablation ladder requiring no architectural surgery.

Arm	Isolates
<code>last_layer</code>	no deep supervision at all
<code>deep_supervision</code>	per-scale losses, uniform weights
<code>mutation</code>	+ logit mixing, unweighted
<code>lomix</code>	+ learnable weights

`mutation` $\rightarrow$ `lomix` is the load-bearing comparison: it isolates the learned weighting from the mixing itself. If the gain came only from having more loss terms, that pair would show it.

3 Environment and setup notes

Item	Value
GPU	NVIDIA GeForce GTX 960, 4 GB, capability 5.2 (Maxwell, no tensor cores)
Driver / CUDA	582.28 / CUDA 13.0 runtime; torch built against 12.1
OS	Windows 11 Pro 26200
Python / PyTorch	3.10.21 / 2.2.2+cu121
timm	0.6.12
Env recipe	<code>setup/01_create_env.ps1</code> , verified by <code>setup/verify_env.py</code>

Two deliberate departures from the repository README:

- **torch 2.2.2+cu121 rather than 1.11.0+cu113.** The wheel ships `sm_50` cubins, which run on this card's `sm_52`; verified by `torch.cuda.get_arch_list()` plus a live `fp32/fp16` matmul.
- **A dedicated conda environment.** The machine's base environment carries `timm` 1.0.26, where `timm.models.layers.trunc_normal_tf_` and `timm.models.helpers.named_apply` have moved. The repository imports both.

An ordering trap worth recording: installing the repository's dependencies *after* the pinned `torch` lets `pip` re-resolve the unpinned `torch` requirement of `timm/thop/ptflops/torchprofile` and silently replace `2.2.2+cu121` with a CPU build, leaving the `cu121` CUDA DLLs in place. The result is two `torch` dist-infos and `OSError: [WinError 127] ...c10_cuda.dll at import`. The setup script installs `torch` last and pins the `torch`-dependent tools with `--no-deps`.

4 Deviations from the paper’s protocol

Deviation	Paper	Here	Effect
Per-step batch	6	2	None on the loss; accumulation restores the effective batch
Grad. accumulation	none	3 (eff. batch 6)	Optimiser sees the paper’s batch; BatchNorm statistics come from 2 samples, not 6
Precision	fp32	fp32	None (AMP measured 2.8–3.2× <i>slower</i> here, §5.2)
Epochs	300	20	Under-trained — the main caveat on every Dice below
Validation	every epoch	every 2 epochs	Coarser best-checkpoint selection
Dataloader workers	8	2	Throughput only
Preprocessing	—	authors’ mirror	The 14→9 label remap is correct as shipped and was left untouched; labels verified to span exactly 0–8 over 200 slices

Everything here is a **reduced-budget reproduction at 20 of the paper’s 300 epochs**. No Dice below should be compared with a published number without repeating that sentence.

Patch applied by `tools/apply_low_vram_patch.py`, which rebuilds from `trainer.py.orig` every time and is reversible with `--revert`. The LoMix loss module, the learnable weights, the operation set and the network are untouched.

5 Results

5.1 Efficiency — the zero-inference-overhead claim holds

224×224, batch 1, 50 timed iterations after 10 warm-up. Source: `logs/bench_efficiency_Ishad.json`.

Encoder	Params (M)	Dec. (M)	GMACs	Latency (ms)	img/s	Peak VRAM
pvt_v2_b0	3.921	0.507	0.657	17.70 ± 2.75	56.5	34.1 MiB
pvt_v2_b2	26.773	1.914	4.324	36.77 ± 7.08	27.2	147.0 MiB

The mixing weights are parameters of the loss module, not of EMCADNet, so all four supervision arms share a byte-identical inference graph and one measurement settles the claim for all of them. The decoder’s 1.914 M parameters match EMCAD’s published decoder cost — a useful check that the model is built as intended. Comparison against the paper’s own cost figures: **[FILL from the LoMix paper — parameter counts and GMACs are comparable; latency on a Maxwell card is not]**.

5.2 What fits in 4 GB

Real forward + LoMix loss + backward + AdamW step on synthetic data, each trial in its own subprocess. Source: `logs/memfit_Ishad_lomix_sweep.json`. The sweep was run twice; **peak memory was identical to the byte, step time was not**.

Encoder	Batch	AMP	Peak VRAM	Step (r1)	Step (r2)	Note
pvt_v2_b0	2	off	1095.9	1034	936	
pvt_v2_b0	4	off	2130.2	1787	1709	
pvt_v2_b0	6	off	3180.1	6558	5870	over TDR watchdog
pvt_v2_b0	2	on	1047.9	3288	3321	AMP $\sim 3.2\times$ slower
pvt_v2_b2	1	off	978.5	640	779	
pvt_v2_b2	2	off	1632.9	1046	1415	chosen
pvt_v2_b2	3	off	2289.6	1432	2193	r2 crosses the watchdog
pvt_v2_b2	4	off	2941.4	3472	5398	over TDR watchdog
pvt_v2_b2	6	off	4268.3	7967	11144	>4096 MiB, WDDM paging
pvt_v2_b2	2	on	1513.8	2940	3969	AMP $\sim 2.8\times$ slower
pvt_v2_b2	4	on	—	—	—	CUDNN_STATUS_EXECUTION_FAILED

VRAM in MiB, step time in ms.

Chosen: pvt_v2_b2, batch 2, accumulation 3 (effective batch 6), AMP off. Four measured reasons:

1. **AMP is a net loss on this card.** No tensor cores, so fp16 convolution buys $\sim 5\%$ memory for a $2.8\text{--}3.2\times$ step-time penalty — the opposite of the usual advice, and only measurement shows it.
2. **The binding constraint is the Windows TDR watchdog, not VRAM.** This is the display GPU; a kernel over $\sim 2\text{s}$ is killed and the CUDA context destroyed. Batch 4 crosses it in both runs.
3. **Step timing is not reproducible to better than $\sim 50\%$** on a machine whose GPU also drives the desktop. Batch 3 measured 1432ms and 2193ms minutes apart with identical peak memory, so it was rejected despite fitting.
4. **Encoder size barely matters:** pvt_v2_b0 and pvt_v2_b2 reach similar throughput, because the loss — not the encoder — dominates the step (§5.3).

The batch-6 row reporting 4268 MiB on a 4096 MiB card is not an error: WDDM allows oversubscription into host RAM, so it “fits” while thrashing.

5.3 Training cost of the supervision, which the paper does not report

Same network, same batch, same data; only `--supervision` differs (pvt_v2_b2, 224^2 , batch 3, AMP off, `logs/memfit_Ishad_*.json`):

Arm	Peak train VRAM	Step time	vs last_layer
last_layer	1008.2 MiB	649 ms	1.00×
deep_supervision	1053.7 MiB	708 ms	1.09×
mutation	1268.3 MiB	938 ms	1.45×
lomix	2289.6 MiB	1649 ms	2.54×

LoMix costs $2.54\times$ step time and $2.27\times$ training memory over a single-logit baseline while adding nothing at inference, and roughly two thirds of that gap comes from the learnable-weight arm alone (`mutation` $\rightarrow$ `lomix`, $938 \rightarrow 1649$ ms). The paper reports the inference cost; this training cost is not reported, and on constrained hardware it decides whether the method is runnable at all — the same class of concern the group’s efficiency work targets, applied to training rather than inference.

5.4 The supervision ablation

Four arms, one network, one dataset, one schedule. 20 epochs, effective batch 6, seed 2222, validation every 2 epochs.

Arm	Isolates	Best Dice	Peak VRAM	Wall	Status
last_layer	no deep supervision	0.6764	892.7	364	completed
deep_supervision	uniform per-scale	0.7979	924.0	368	completed
mutation	mixing, unweighted	0.7788	1067.0	405	completed
lomix	learnable weights	0.7225 [†]	1768.8	594	diverged (NaN) ep. 13

VRAM in MiB, wall clock in minutes. [†]best value before divergence.

Validation curves (Dice at each validation):

Arm	Curve
last_layer	0.676, 0.588, 0.562, 0.593, 0.569, 0.605, 0.566, 0.578, 0.618, 0.598
deep_supervision	0.659, 0.613, 0.607, 0.684, 0.626, 0.727, 0.759, 0.751, 0.741, 0.798
mutation	0.563, 0.643, 0.698, 0.701, 0.683, 0.768, 0.772, 0.757, 0.773, 0.779
lomix	0.609, 0.676, 0.708, 0.703, 0.7225 , 0.711, 0.000, 0.000

Two things are visible immediately. `last_layer` peaks at its *first* validation and then decays, while the supervised arms climb steadily — consistent with the training-mode defect in §6.4. And at this budget the paper’s ordering does **not** reproduce: uniform deep supervision beats unweighted mixing, which beats the learnable-weight arm, which then diverges. §7 discusses why, and §8 lists what would settle it.

5.5 The lomix arm diverged

At iteration 14900, epoch 13, `loss`, `deep_supervision_loss` and `mutation_loss` all became NaN simultaneously. Every subsequent validation returned Dice 0.000. The best pre-divergence value was 0.7225 at epoch 9.

This was not a hardware event: fp32 throughout, no OOM, no watchdog reset, and the weights in `best.pth` are all finite. The cause is in the loss (§6.2–§6.3). An earlier `lomix` attempt survived to epoch 19 only because a machine shutdown had forced a restart at epoch 11 with a fresh optimizer, which reset the state that was accumulating toward overflow — that run is reported separately as **not comparable**, because a resume without optimizer state is a different experiment.

5.6 Learned mixing weights

All weights initialise at `softplus(0) = 0.6931`. Final values from the `lomix` arm before divergence:

Group	Value	Movement from initialisation
Per-scale p_4, p_3, p_2, p_1	0.379, 0.386, 0.392, 0.400	all fell to ≈ 0.39 , nearly uniform
mul (11 subsets)	0.434–0.464	up-weighted
add (11 subsets)	0.417–0.459	up-weighted
wf (11 subsets)	0.385–0.397	mid
concat (11 subsets)	0.3755–0.3770	collapsed, identical across all 11

The per-scale weights barely differentiate at 20 epochs, which suggests LoMix’s weighting needs far more budget to specialise than a reduced run provides. The `concat` collapse turned out to have a concrete cause — §6.3.

6 Defects found in the released codebase

All were found by running the code, and all are reproducible.

6.1 The training script cannot start

`train_synapse_lomix.py:10` imports `PVT_CASCADE`, which is defined nowhere in the repository — `lib/networks.py` defines only `EMCADNet`, and the only other reference is a commented-out model line. As shipped, the script fails at import before parsing an argument. `EMCAD`’s equivalent script imports only `EMCADNet`, so this looks like leftover copy-paste.

6.2 `mul` is numerically unbounded

It multiplies raw logits, so a k -map combination is a k -fold product growing like $|\text{logit}|^k$. Measured on the trained checkpoint’s own logits ($\max |p| = 6.16$), via `tools/diagnose_loss_ops.py`:

Operation	$k = 2$	$k = 3$	$k = 4$
add	12.0	17.9	23.6
mul	36.2	211.0	1214.0
wf	6.1	6.1	5.9
concat	2.8	2.9	1.9

Cross-entropy and Dice saturate on values of that size and the gradient overflows. Nothing clips or normalises the product. This is the mechanism behind the epoch-13 NaN, and it is a property of the method as released, not of this hardware — the magnitudes above are computed on the model’s own trained logits.

6.3 `concat` rebuilds a random convolution on every forward pass

In `trainer.py`:

```
elif op == 'concat':
    cat = torch.cat([output_maps[c] for c in comb], dim=1)
    conv = nn.Conv2d(cat.size(1), self.num_classes, kernel_size=1).to(device)
    mutated = conv(cat)
```

The `conv` is created inside `forward`, randomly initialised each call, never registered as a submodule and never given to the optimizer. Evidence: calling the loss twice with identical inputs under `no_grad` gives

Operation	Max $ a - b $
add	0.0 deterministic
mul	0.0 deterministic
wf	0.0 deterministic
concat	5.41 non-deterministic

and none of the loss module’s 66 parameter tensors belongs to it (the `conv` parameters that *are* registered all belong to `weighted_fusion_modules`, i.e. the `wf` operation). So the `concat` branch cannot learn and contributes a fresh random projection each step. This explains §5.6 exactly: `concat`’s learnable weights have no gradient signal to differentiate on, so they collapse to a common value.

Taken with §6.2: of four mixing operations, one injects unbounded values and one injects random noise. Any claim about *which* mixtures matter rests on an operation set that is half broken.

6.4 Training continues in eval mode after the first validation

`trainer.py` calls `model.train()` once *before* the epoch loop; `inference()` calls `model.eval()` and runs at the end of every epoch; nothing switches back. From epoch 1 onward the decoder’s `BatchNorm2d` layers normalise with frozen running statistics and stop updating them, and dropout/drop-path are inactive. The same pattern is present in the EMCAD and G-CASCADE trainers.

This was **left intact** — the published results were produced by this code, so preserving the behaviour is what replication means. The patch adds `--restore_train_mode` (default off) so the alternative can be run as a separately labelled experiment. Not run here for lack of GPU time.

6.5 Windows portability

`worker_init_fn` is defined as a local function inside `trainer_synapse`. Windows starts DataLoader workers with `spawn`, which pickles the callable, and local functions are not picklable: `AttributeError: Can’t pickle local object 'trainer_synapse.<locals>.worker_init_fn'`. Linux uses `fork` and is unaffected. Fixed by hoisting the function to module scope and binding the seed with `functools.partial`; seeding behaviour is unchanged.

7 Limitations of this replication

Single seed, 20 of 300 epochs, one dataset, one encoder, no statistical testing. The `deep_supervision/mutation` gap (0.798 vs 0.779) is well inside what one seed can produce.

What these runs **do** support independently of budget: the inference-cost claim (measured directly), the training-cost ratios (measured under identical conditions), and the two loss defects (verified on the model’s own logits, not dependent on training length). What they do **not** settle is whether LoMix outperforms uniform deep supervision at full budget — that needs the 300-epoch run this hardware cannot deliver.

8 Discrepancies and what would settle them

1. **The ordering does not reproduce at 20 epochs.** `deep_supervision` (0.798) > `mutation` (0.779) > `lomix` (0.723, diverged). Most likely under-training: §5.6 shows the mixing weights had barely differentiated from their common initialisation, so the mechanism that is supposed to create LoMix’s advantage had not yet engaged. Settled by a full 300-epoch run.
2. **The lomix arm has no completed run.** It needs either gradient clipping or a bounded `mul` (e.g. multiplying softmax probabilities rather than raw logits) to finish. Either is a change to the method and would have to be applied to every arm to keep the comparison fair.
3. **Single seed, one dataset, one encoder, no statistical testing.**
4. Paper reference values for orientation: **[FILL from the LoMix paper, clearly labelled as the paper’s 300-epoch numbers, not results of these runs].**

9 Where I would take it

Two directions follow directly from the findings, and the first is my Step 2 proposal.

Uncertainty-conditioned mixing weights. LoMix learns one global answer to “which scale do I trust”, then freezes it. §5.6 shows those weights differentiate very slowly; §5.3 shows enumerating all 44 combinations is what makes training expensive. Conditioning the weights on a per-sample uncertainty estimate addresses both: it gives the weights a richer signal than a single global scalar, and it offers a principled basis for *selecting* which subsets are worth

supervising rather than enumerating all of them — cutting cost on precisely the axis that dominates. Because the weights live in the loss, the inference graph stays untouched, preserving the property that makes LoMix deployable. Evaluated on calibration (ECE, reliability) as well as Dice, this connects directly to the calibration-driven selection in CLEAR-MoE and the confidence-aware objective in my LiDAR detection work.

Bounded mixing operations. Multiplying softmax probabilities rather than raw logits, or normalising the product, would remove the divergence mode in §6.2 at no representational cost — and registering the `concat` conv properly would make a quarter of the operation set functional for the first time. Both are small changes with a measurable prediction attached: if `concat` is currently inert, fixing it should change its learned weights away from the collapsed value.

10 Reproducing this

```
powershell -ExecutionPolicy Bypass -File setup\01_create_env.ps1
conda activate lomix
cd repos\LoMix
python ../../tools/bench_efficiency.py
python ../../tools/memfit.py
cd ../../
python tools/check_data.py --repo repos\LoMix          # must print DATA OK
python tools/apply_low_vram_patch.py --repo repos\LoMix
.\run_ablation.ps1                                     # ~18 h, 4 arms
python tools/collect_results.py --stamp <stamp>
python tools\diagnose_loss_ops.py --ckpt <best.pth>    # the section 6.2-6.3 evidence
```

Attached: raw logs and manifests (logs/), the patch script and `trainer.py.orig` for a line-level diff, and the diagnostic tool.

Runs are checkpointed after every epoch with model, optimizer, scaler, `best_dice`, `iter_num` and epoch, written atomically, so `-Resume` continues a shutdown-interrupted run identically rather than restarting the optimizer.

Appendix — slide outline (6 slides)

1. **LoMix in one slide** — decoder logits → subset mixing → learned weights; loss-only, therefore zero inference overhead.
2. **Why this paper** — CLEAR-MoE (calibration-driven selection) and PSC (self-conditioning judged on calibration) ask the same question.
3. **Setup** — four arms, one network; the 4 GB deviation table; AMP measured slower, TDR watchdog as the real constraint.
4. **What reproduced** — efficiency table; training-cost table (2.54×).
5. **What did not** — ablation table, divergence at epoch 13, and the two loss defects with the `mul` magnitude and `concat` determinism evidence.
6. **Next** — uncertainty-conditioned weights, bounded operations → Step 2 proposal.